Kaggle Competition Report

Irrigation Need Prediction - Solution 5 Pipeline

Student Information	
Name	Sana Munir Alam
Roll No	24K-0573
Section	BCS-4E
Submission Date	27-04-2026

Item	Details
Source code	Solution5.py
Main approach	Feature engineering + 5-fold CV + LightGBM, XGBoost, CatBoost + soft voting/stacking + multi-seed averaging
Kaggle score	0.96198
Leaderboard rank	992

1. Competition Overview

Field	Description
Competition Name	Irrigation Need Prediction / Irrigation Need Classification
Problem Type	Multiclass classification
Target Variable	Irrigation_Need with three classes: Low, Medium, and High
Evaluation Metric	Balanced accuracy score, which is suitable because it gives equal importance to each class and reduces the effect of class imbalance.
Dataset Understanding	The dataset contains soil, weather, crop, irrigation, season, water-source, region, and field-size information. The task is to predict how much irrigation is needed for each test record. The script reads train.csv, test.csv, and sample_submission.csv, then creates a Kaggle-ready submission.csv file.

2. Data Preprocessing

The preprocessing pipeline was separated into two branches so that different model families receive suitable input representations.

Step	Implementation in Solution 5
Missing values handling	The script checks and prints total missing values in the training set. No explicit imputation is applied, so the pipeline assumes the provided competition files are already clean or contain no problematic missing values.
Target encoding	The target classes are encoded into numeric labels. The intended class order is Low, Medium, High.
Categorical encoding	For sklearn and tree models, categorical columns are encoded with OrdinalEncoder using unknown_value = -1 for unseen categories. CatBoost receives the raw categorical columns and uses cat_features internally.
Feature scaling	A StandardScaler is used for numerical columns in the sklearn preprocessing branch. For tree-based models such as XGBoost and LightGBM, numerical features are passed through without scaling.
Train-validation split	5-fold StratifiedKFold with shuffle=True and random_state=42 is used. Stratification keeps the class distribution similar across folds.
Class imbalance handling	Balanced class weights are computed using compute_class_weight. These weights are converted into sample weights and passed to models that support sample_weight.

Feature Engineering Section

The pipeline adds 18 engineered numeric features. These features model irrigation stress, water availability, soil condition, area effect, and weather interactions.

Feature Group	Examples
Evaporation/weather stress	evapo_proxy, wind_drying, sunshine_stress, heat_index
Moisture stress	moisture_deficit, temp_moisture_stress, moisture_sq
Water availability	water_balance, net_water_avail, rain_moisture
Soil quality/stress	soil_health, salinity_stress
Area-adjusted indicators	rainfall_per_ha, prev_irr_per_ha, area_log
Interactions/proxies	moisture_wind, temp_wind, need_proxy

After engineering, the final feature set contains 8 categorical features and 29 numerical features, making 37 total model inputs.

3. Models Attempted

Model	Purpose / Configuration
Decision Tree	Baseline supervised model using max_depth=15, min_samples_leaf=10, and class_weight=balanced.
Gaussian Naive Bayes	Probabilistic baseline. Useful for comparison, but limited because features are not independent and many relationships are nonlinear.
K-Means as classifier	Unsupervised baseline using 3 clusters on up to 30,000 samples. Clusters are mapped to the majority true class.
Logistic Regression	Linear supervised baseline using lbfgs, max_iter=1000, C=1.0, and class_weight=balanced.
Linear Regression OvR	Custom one-vs-rest linear regression classifier used to satisfy the linear-model requirement.
Random Forest	Tree ensemble baseline with 200 estimators, max_depth=20, balanced_subsample weights, and CV on up to 100,000 samples.
XGBoost	Advanced gradient boosting model with 500 estimators, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, and histogram tree method.
LightGBM	Advanced gradient boosting model with 1000 estimators, num_leaves=127, learning_rate=0.03, class_weight=balanced, and multiclass objective.
CatBoost	Advanced boosting model with 800 iterations, depth=6, learning_rate=0.05, class weights, and native handling of categorical columns.
Soft-voting ensemble	Combines probability outputs from LightGBM, XGBoost, and CatBoost using weights 0.40, 0.35, and 0.25 respectively.
Stacking layer	Uses out-of-fold probabilities from the three boosting models as meta-features. A Logistic Regression meta-learner is trained on these OOF features.

4. Cross Validation Strategy

The script evaluates models using 5-fold StratifiedKFold. For each fold, the model is trained on four folds and validated on the remaining fold. Out-of-fold predictions are stored for all supervised models so that balanced accuracy can be measured on unseen training rows.

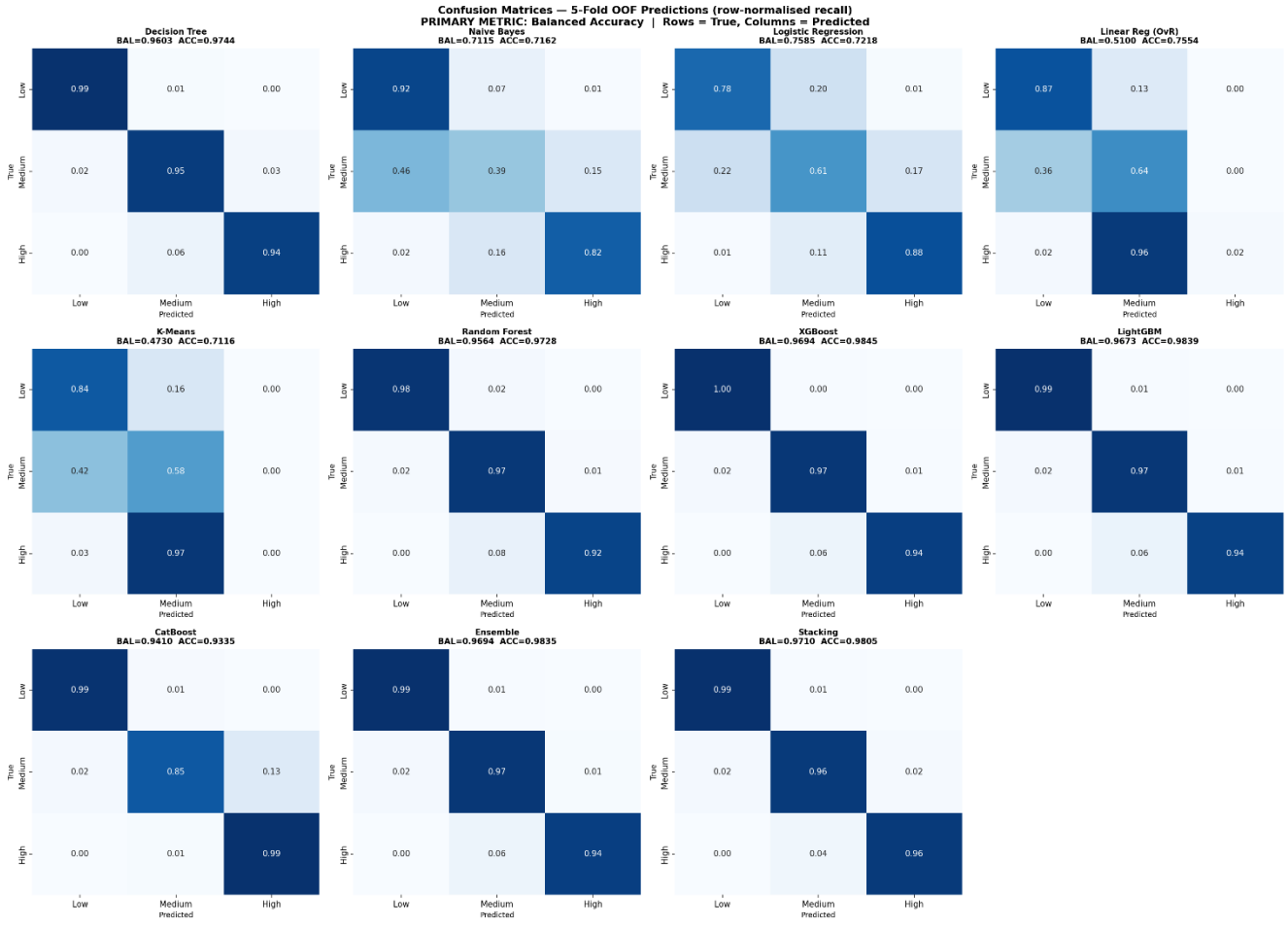
Item	Details
K-Fold details	5 folds, stratified by target class, shuffled with random_state=42.
Primary metric	balanced_accuracy_score. Regular accuracy is also printed for reference but is not the main selection metric.
OOF predictions	OOF predictions are stored for each model. For stacking, the meta-learner is trained using OOF probability features only, which reduces leakage risk.

Item	Details
Same CV splits	The same StratifiedKFold object is reused across the base models, soft-voting ensemble, and stacking evaluation.
LOOCV observations	Leave-One-Out CV was not used because the dataset is large and the pipeline includes heavy models. LOOCV would require too many fits and would not be computationally practical.
Best validation accuracy	The exact best CV value is generated when the code is run and saved in outputs/summary.json. The strongest validation candidates are the LightGBM/XGBoost/CatBoost ensemble and the stacking model.

5. Failed Attempts and Insights

Model used	Accuracy / Result	What went wrong?	What was improved?
K-Means classifier	Expected to be weak; script reports balanced accuracy on a 30k sample.	It is unsupervised and does not learn the target boundary directly. Cluster-to-class mapping is too simple for this problem.	Used only as a required baseline. Final solution moved to supervised boosting models.
Gaussian Naive Bayes	Used as a baseline; exact value is written in outputs/classification_reports.txt after running.	Assumes independent Gaussian features, but irrigation need depends on feature interactions such as temperature, moisture, rainfall, and crop type.	Added engineered interaction features and stronger nonlinear models.
Linear / Logistic Regression	Useful baseline, but not expected to match tree boosting.	Linear decision boundaries cannot fully capture nonlinear irrigation relationships.	Used gradient boosted trees and stacking to capture nonlinear effects.
Single Decision Tree	Baseline only.	Single trees can overfit and are less stable than ensembles.	Used Random Forest and gradient boosting for better generalization.
Random Forest	Stronger than simple baselines, but tested on a 100k subsample for speed.	Less competitive and slower compared with boosting on structured tabular data.	Final model used LightGBM, XGBoost, and CatBoost.

Confusion matrix screenshot: the code automatically generates outputs/confusion_matrices.png.



6. Final Model Selection

The final selected system is a high-performance ensemble/stacking pipeline based on LightGBM, XGBoost, and CatBoost. The code compares the soft-voting ensemble against the stacking model using OOF balanced accuracy and then chooses the better-performing path for submission.

Component	Final Configuration
Base models	LightGBM, XGBoost, and CatBoost
Meta model	Logistic Regression with lbfgs solver, max_iter=2000, C=1.0, and class_weight=balanced
Stacking features	OOF predicted probabilities from base models. With three base models and three classes, the meta-feature matrix has 9 columns.
Final averaging	Five random seeds are used: 42, 52, 62, 72, and 82. Test probabilities are averaged across seeds to reduce variance.
Submission logic	If stacking OOF balanced accuracy is higher than soft-voting OOF balanced accuracy, stacking is used. Otherwise, the soft-voting ensemble is used.

Important Hyperparameters

Model	Main hyperparameters
LightGBM final	n_estimators=1500, num_leaves=127, learning_rate=0.025, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0, min_child_samples=20
Model	Main hyperparameters
XGBoost final	n_estimators=600, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0, tree_method=hist
CatBoost final	iterations=800, depth=6, learning_rate=0.05, l2_leaf_reg=3.0, loss_function=MultiClass, class_weights=balanced weights
Soft-vote weights	LightGBM=0.40, XGBoost=0.35, CatBoost=0.25 when CatBoost is available.

Why this model was selected:

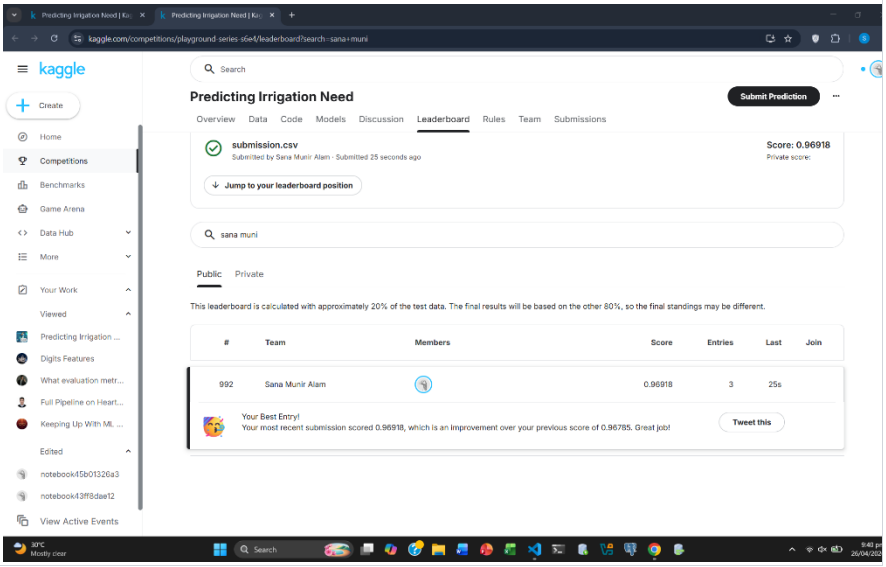
• Boosting models perform strongly on structured tabular data and capture nonlinear relationships between soil, weather, crop, and irrigation variables.

• OOF stacking allows the meta-learner to learn which base model is more reliable for different regions of the probability space.

• Balanced class weights and sample weights improve performance on minority classes, which matters because balanced accuracy treats all classes equally.

• Multi-seed averaging makes the final predictions more stable and reduces randomness from model initialization.

7. Leaderboard Performance

Item	Value
Kaggle score	0.96198
Leaderboard rank	992
Submission file	outputs/submission.csv
Screenshot attached	

8. Conclusion and Learnings

This solution progressed from simple baseline models to a stronger ensemble and stacking approach. The main improvement came from combining domain-based feature engineering with class-balanced gradient boosting models.

Area	Learning / Insight
Key insights	Irrigation need depends on interactions between moisture, rainfall, temperature, wind, crop, and soil variables. Engineered stress and water-balance features helped expose these relationships.
Challenges faced	The target classes can be imbalanced, so regular accuracy alone can be misleading. Heavy models also increase computation time, especially with cross-validation and multi-seed averaging.
What worked best	The strongest approach was the LightGBM, XGBoost, and CatBoost probability ensemble/stacking system evaluated with balanced accuracy.
Future improvements	Use Optuna or Bayesian optimization for model hyperparameters, calibrate probabilities, test additional stacking meta-learners, perform leakage-safe fold-wise preprocessing, and attach full CV artifacts to the final report.

Generated Output Files from the Script

File	Purpose
outputs/submission.csv	Final Kaggle submission file.
outputs/confusion_matrices.png	Row-normalized confusion matrices for all evaluated models.
outputs/model_comparison.png	Balanced accuracy comparison chart across models.
outputs/feature_importance.png	Top 20 LightGBM feature importances.
outputs/classification_reports.txt	Per-model precision, recall, F1-score, accuracy, and balanced accuracy reports.
outputs/summary.json	Compact JSON summary of balanced accuracy and accuracy metrics.

Overall, the pipeline is a complete Kaggle workflow: it loads the data, engineers features, validates multiple baseline and advanced models, generates diagnostic plots, selects between stacking and soft voting, trains a multi-seed final model, and writes a valid submission file.